APUNTES 02. MODULO 06. proyectos

Objetivo del módulo

Identificar las distintas fases que atraviesa un proyecto software real desde su planteamiento hasta su desmantelamiento, y las principales metodologías utilizadas para su correcta gestión.

Resultados de aprendizaje

• Reconocer y aplicar las fases de un proyecto (requisitos → mantenimiento).

• Diferenciar metodologías monolíticas (predictivas) y metodologías ágiles (iterativas).

Índice

1. Visión general del ciclo de vida

2. Fases de un proyecto software

  2.1 Toma de requisitos

  2.2 Análisis

  2.3 Diseño

  2.4 Implementación

  2.5 Pruebas

  2.6 Lanzamiento

  2.7 Mantenimiento

3. Artefactos, roles y entregables típicos

4. Metodologías de gestión

  4.1 Metodologías monolíticas (predictivas)

  4.2 Metodologías ágiles

  4.3 Comparativa rápida y cuándo elegir cada una

5. Caso práctico guiado (mini-proyecto)

6. Actividades y ejercicios

7. Checklist final y mini test

8. Glosario básico

1. Visión general del ciclo de vida

Un proyecto software no es solo "programar". Es un proceso completo que empieza cuando alguien plantea una necesidad ("quiero una app que…") y termina cuando el sistema se retira o se sustituye. El ciclo de vida describe esas etapas y ayuda a planificar trabajo, costes, tiempos y calidad.

Idea clave

Cuanto antes detectes un problema (requisitos mal entendidos, un diseño pobre, un bug crítico), más barato y rápido es corregirlo. Por eso las fases existen: para reducir riesgos.

Mapa mental rápido (de principio a fin)

Requisitos → Análisis → Diseño → Implementación → Pruebas → Lanzamiento → Mantenimiento

---

2. Fases de un proyecto software

A continuación verás cada fase con: objetivo, actividades típicas, entregables y un ejemplo sencillo. Piensa en un mini-proyecto: una web para gestionar "Tareas" (crear, listar, completar).

2.1 Toma de requisitos

Objetivo:

Entender qué necesita el cliente/usuario, qué problema quiere resolver y qué límites existen (tiempo, presupuesto, tecnología).

Actividades típicas:

- Reuniones/entrevistas con cliente y usuarios.
- Análisis del contexto (quién usa el sistema, para qué, con qué datos).
- Priorizar necesidades (qué es imprescindible y qué es deseable).
- Aclarar criterios de aceptación (cómo sabremos que está "bien hecho").

Entregables habituales:

- Documento de requisitos (o backlog inicial).
- Historias de usuario / casos de uso.
- Alcance (qué entra y qué NO entra).

Ejemplo práctico:

1. El cliente pide: "Quiero crear tareas con título y fecha límite; ver una lista; marcar como completadas; y filtrar por completadas/no completadas."

2.2 Análisis

2.3 Diseño

Objetivo:

Decidir cómo se construirá el sistema: arquitectura, componentes, interfaces y decisiones técnicas.

Actividades típicas:

- Elegir arquitectura (por ejemplo, MVC, capas, API + Front…).
- Diseñar la base de datos (tablas, claves, índices).
- Diseñar la interfaz (pantallas, navegación).
- Diseñar contratos entre componentes (métodos, DTOs, endpoints).

Entregables habituales:

- Diagrama de arquitectura (capas / módulos).
- Diseño de BD (modelo relacional).
- Prototipos UI / wireframes.
- Decisiones técnicas documentadas (stack, librerías, seguridad).

Ejemplo práctico:

3. Para .NET: decidimos una web ASP.NET MVC con una capa de datos (Entity Framework) y una tabla Tareas.

2.4 Implementación

Objetivo:

Construir el software: programar, integrar componentes, versionar y revisar el código.

Actividades típicas:

- Crear estructura del proyecto y repositorio (Git).
- Implementar funcionalidades por módulos.
- Revisiones de código (code review) y estándares (nombres, formato).
- Integración continua (compilar, ejecutar pruebas automáticamente).

Entregables habituales:

- Código fuente versionado.
- Builds ejecutables / desplegables.
- Documentación técnica mínima (README, configuración).

Ejemplo práctico:

4. Implementamos: controlador TareasController, vistas Index/Create/Edit, y el modelo Tarea con validaciones.

2.5 Pruebas

Objetivo:

Comprobar que el software hace lo que debe, sin romperse, y con calidad suficiente antes de entregar.

Actividades típicas:

- Pruebas unitarias (métodos y reglas).
- Pruebas de integración (BD, servicios, APIs).
- Pruebas de sistema/aceptación (flujo completo como usuario).
- Pruebas no funcionales (rendimiento, seguridad, usabilidad).

Entregables habituales:

- Plan de pruebas (qué se prueba y cómo).
- Evidencias (capturas, informes, resultados automáticos).
- Listado de incidencias (bugs) y estado.

Ejemplo práctico:

5. Caso: “Crear tarea sin título” debe mostrar error de validación y NO guardar en BD.

2.6 Lanzamiento (despliegue)

Objetivo:

Poner el software en producción o en el entorno del cliente de forma controlada.

Actividades típicas:

- Preparar entorno (servidor, BD, variables de configuración).
- Desplegar (publicación) y validar con “smoke tests” (pruebas rápidas).
- Formación básica a usuarios y manual de uso.
- Plan de rollback (cómo volver atrás si algo sale mal).

Entregables habituales:

- Versión desplegada (release).
- Notas de versión (qué incluye, cambios, known issues).
- Manual de usuario (básico) y guía de soporte.

Ejemplo práctico:

6. Publicamos la web en IIS/Azure, aplicamos migraciones de BD y verificamos: login, crear, listar y editar tarea.

2.7 Mantenimiento

Objetivo:

Corregir errores, mejorar funcionalidades y adaptar el sistema a cambios (legales, técnicos o de negocio).

Actividades típicas:

- Mantenimiento correctivo (bugs).
- Mantenimiento evolutivo (nuevas funciones).
- Mantenimiento adaptativo (cambios de entorno: SO, librerías, BD).
- Monitorización y soporte (logs, métricas, alertas).

Entregables habituales:

- Backlog de incidencias/mejoras.
- Parches y nuevas versiones.
- Documentación actualizada.

Ejemplo práctico:

7. Añadimos: recordatorios por email y un filtro por “vencidas”. Revisamos seguridad y rendimiento tras el cambio.

Nota sobre “desmantelamiento”

Cuando un sistema se sustituye o deja de usarse, hay que planificar su retirada: migrar datos, desactivar accesos, archivar documentación, cumplir normativa (por ejemplo, conservación/borrado) y comunicar a usuarios. Esto evita pérdidas de información y problemas legales.

3. Artefactos, roles y entregables típicos

Para que un proyecto avance sin caos, se suele trabajar con artefactos (documentos/elementos) y roles claros. En proyectos pequeños una persona puede asumir varios roles.

3.1 Roles frecuentes

Rol	Qué hace (resumen)
Cliente/Usuario	Define necesidades, valida entregas, prioriza.
Product Owner (en ágil)	Mantiene backlog y prioridades, representa al negocio.
Project Manager / Scrum Master	Planifica, elimina bloqueos, coordina y protege el proceso.
Analista	Detalle de requisitos, modelado y reglas de negocio.
Desarrollador/a	Implementa funcionalidades y corrige incidencias.
QA / Tester	Define y ejecuta pruebas, reporta defectos, asegura calidad.
DevOps / Sistemas	Infraestructura, despliegue, monitorización, CI/CD.

3.2 Entregables por fase (vista rápida)

Fase	Entregable típico	Ejemplo (mini-proyecto tareas)
Requisitos	Historias / documento	“Crear tarea, listar, completar” + criterios
Análisis	Modelo de datos y reglas	Entidad Tarea + validaciones
Diseño	Arquitectura y UI	MVC + wireframe de pantallas
Implementación	Código + build	Controlador/Vistas/EF
Pruebas	Plan + evidencias	Casos: título obligatorio, filtros

Lanzamiento	Release + notas	v1.0 en servidor + smoke tests
Mantenimiento	Backlog + parches	v1.1 recordatorios + bugfix

4. Metodologías de gestión de un proyecto software

Una metodología es una forma organizada de trabajar: cómo planificamos, cómo entregamos, cómo medimos avance y cómo gestionamos cambios. Aquí veremos dos familias: predictivas (monolíticas) y ágiles.

4.1 Metodologías monolíticas

Se llaman "predictivas" porque intentan definir al inicio el alcance, coste y calendario con bastante detalle. El ejemplo clásico es el modelo en cascada (Waterfall).

Características típicas:

- Fases más secuenciales (una termina y empieza la siguiente).
- Mucha documentación inicial (requisitos/diseño).
- Cambios más costosos: se gestionan con control formal (peticiones de cambio).
- Útil cuando los requisitos están muy claros y el entorno es estable.

Ventajas / Inconvenientes

Ventajas:

- Facilita estimar si todo está bien definido.
- Buena trazabilidad documental (auditorías, normativa).
- A veces encaja con contratos cerrados.

Inconvenientes:

- Si cambian requisitos, rehacer documentación y diseño puede ser caro.
- El usuario ve el producto tarde (riesgo de "no era lo que quería").
- Puede generar entregas grandes y poco frecuentes.

4.2 Metodologías ágiles

Las metodologías ágiles trabajan en iteraciones cortas para entregar valor pronto y aprender con feedback. La idea es adaptarse al cambio en lugar de evitarlo. Un ejemplo muy usado es Scrum.

Conceptos clave en ágil:

- Iteraciones (sprints) de 1–4 semanas.
- Backlog priorizado: lo más importante primero.
- Reuniones cortas y frecuentes para inspeccionar y adaptar.
- Entrega incremental: versiones pequeñas pero funcionales.

Ejemplo de sprint (mini-proyecto tareas)

8. Sprint 1: listar tareas + crear tarea (validación título).
9. Sprint 2: editar y completar tareas + filtro completadas.
10. Sprint 3: fechas límite + tareas vencidas + mejoras UI.

4.3 Comparativa rápida y cuándo elegir cada una

Aspecto	Predictiva (monolítica)	Ágil
Planificación	Alta al inicio	Replanificación frecuente
Cambios	Costosos / formales	Esperados / gestionados
Entrega	Al final o en hitos grandes	Frecuente e incremental
Documentación	Más pesada	La justa + útil
Riesgo principal	Construir “algo correcto” pero no útil	Desorden si no hay disciplina
Cuándo encaja	Requisitos estables, normativa, contratos cerrados	Requisitos cambiantes, necesidad de feedback

5. Caso práctico guiado

Vamos a practicar el ciclo de vida con un mini-proyecto realista. No es para programar aquí, sino para aprender qué decisiones tomar en cada fase.

Enunciado

Una academia quiere una aplicación web sencilla para registrar incidencias de aula (proyector roto, falta de material, etc.). Debe permitir: crear incidencia, asignar prioridad (Baja/Media/Alta), marcar como resuelta, y listar/filtrar.

5.1 Requisitos (ejemplo en formato historia de usuario)

- Como profesor, quiero crear una incidencia con título y descripción para dejar constancia del problema.
- Como coordinador, quiero ver incidencias por prioridad para atender primero las urgentes.
- Como administración, quiero marcar incidencias como resueltas para cerrar el trabajo realizado.

Criterios de aceptación (ejemplos)

- No se puede guardar una incidencia sin título.
- La prioridad por defecto es Media.
- Al marcar como resuelta, queda registrada la fecha de resolución.

5.2 Análisis: modelo de datos mínimo

Entidad Incidencia:

- Id (int)
- Titulo (string, obligatorio)
- Descripcion (string, opcional)
- Prioridad (enum: Baja/Media/Alta)
- Resuelta (bool)
- FechaCreacion (DateTime)
- FechaResolucion (DateTime?, opcional)

5.3 Diseño: pantallas y navegación

Pantallas mínimas:

- Listado + filtros (prioridad, resuelta).
- Crear incidencia.
- Editar/Marcar como resuelta.

Decisiones técnicas sugeridas (.NET)

- ASP.NET MVC tres capas (UI, Controller y datos).
- Validaciones
- Salida de Logs para incidencias en producción.

5.4 Implementación: desglose en tareas (ejemplo)

11. Crear proyecto y repositorio.
12. Creación y Migración BD.
13. CRUD básico (Create/Read/Update).
14. Filtros y Listados
15. Botón “Marcar como resuelta” + fecha.
16. Validaciones y mensajes de error.

5.5 Pruebas: casos mínimos

- Crear incidencia sin título → debe fallar validación.
- Crear incidencia con prioridad vacía → prioridad Media por defecto.
- Marcar como resuelta → Resuelta=true y FechaResolucion no nula.
- Filtrar por Alta → solo aparecen incidencias Alta.

5.6 Lanzamiento y mantenimiento

Plan de lanzamiento: desplegar en un servidor, crear usuario admin, ejecutar smoke tests (abrir app, crear incidencia, listar, resolver). Mantenimiento: registrar bugs, priorizar mejoras (por ejemplo, adjuntar fotos).

6. Actividades y ejercicios

Actividad 1 – Identifica fases

Lee estas situaciones y escribe en qué fase encajan mejor (puede haber más de una):

- El cliente dice: “Quiero que también se pueda exportar a Excel”.
- Aparece un bug al guardar en base de datos.
- Se prepara el servidor y las variables de configuración.

Actividad 2 – Mini backlog

Crea 8 historias de usuario para el mini-proyecto de incidencias.

Marca 3 como “Imprescindibles (MVP)”, 3 como “Importantes” y 2 como “Opcionales”.

Actividad 3 – Plan de pruebas básico

Escribe 6 casos de prueba con este formato: ID, Precondición, Pasos, Resultado esperado.

Ejemplo:

PT-01 | Precondición: usuario en pantalla Crear | Pasos: dejar título vacío y pulsar Guardar | Esperado: mensaje de error y no se crea el registro.

Actividad 4 – Riesgos del proyecto

Escribe al menos 5 riesgos y cómo los mitigas. Ejemplos:

- Requisitos cambiantes → usar iteraciones cortas y revisiones frecuentes.
- Falta de acceso al servidor de producción → preparar entorno con antelación y plan B.
- Dependencia de una API externa → mock en desarrollo + reintentos + timeouts.

7. Checklist final y mini test

Checklist de proyecto

☐ Requisitos escritos + alcance claro (qué entra / qué no entra).

☐ Criterios de aceptación definidos para lo importante.

☐ Diseño de datos y arquitectura revisados.

☐ Código versionado y con estructura clara.

☐ Pruebas mínimas ejecutadas (unitarias/integración/sistema).

☐ Plan de despliegue

☐ Plan de mantenimiento (cómo se reportan incidencias, quién responde).

8. Glosario básico

Alcance: Conjunto de funcionalidades/servicios que se entregarán (y sus límites).

Backlog: Lista priorizada de trabajo pendiente (historias, tareas, bugs).

Criterio de aceptación: Condición verificable que confirma que un requisito está cumplido.

Entrega incremental: Publicar mejoras en partes pequeñas y utilizables, no todo al final.

Incidencia (bug): Defecto del software respecto a lo esperado.

MVP: Producto mínimo viable: lo mínimo para aportar valor y obtener feedback.

Release: Versión publicada/desplegada.

Rollback: Volver a una versión anterior si el despliegue falla.

Smoke test: Prueba rápida para comprobar que lo básico funciona tras desplegar.

Trazabilidad: Capacidad de seguir un requisito desde su origen hasta su implementación y pruebas.